



系统编程

Systems Programming

第二章 进程控制



授课时间

第三周

2.1 进程状态

2.2 进程标识

2.3 进程创建

2.4 进程终止

2.5 守护化OSChatBox服务器



本章学习目标：

掌握进程生命周期管理



项目拆解

实现OSChatBox服务器守护进程

项目里程碑

服务器守护化
配置守护进程与日志文件

知识目标

- ❖ 理解进程状态及其转换原理
- ❖ 掌握进程标识符（PID）的获取与运用
- ❖ 剖析进程创建(fork())与终止 (exit())机制
- ❖ 实现OSChatBox的守护化部署

能力目标

- ❖ 监控并管理进程生命周期
- ❖ 创建、终止和回收进程资源
- ❖ 编写健壮的守护进程
- ❖ 调试多进程并发问题



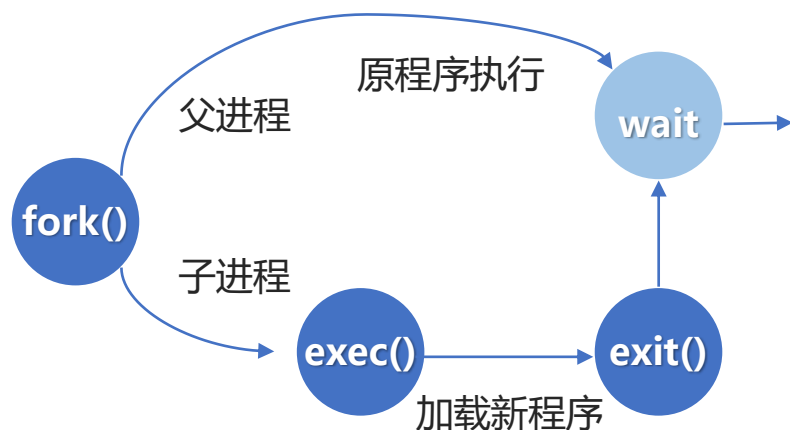
2.3

进程创建 – 2.3.2 程序替换：exec函数族



从程序角度看进程创建

- 程序复制：fork()
 - 父子程序一样
- 程序替换：fork + exec
 - 子进程执行新程序
 - 父进程继续执行原程序



exec() 函数族

```
#include <unistd.h>

extern char **environ;

int execl(const char *path, const char *arg, ..., NULL);

int execlp(const char *file, const char *arg, ...);

int execl_e(const char *path, const char *arg, ..., char *const envp[]);

int execv(const char *path, char *const argv[]);

int execvp(const char *file, char *const argv[]);

int execvpe(const char *file, char *const argv[], char *const envp[]);
```

- path: 可执行文件的路径名
- arg: 参数
- NULL: 参数终结



2.3

进程创建 – 2.3.2 程序替换：exec函数族



```
1  #include <unistd.h>
2  #include <stdio.h>
3  int main(int argc, char *argv[])
4  {
5      int ret = 0;
6      printf("Process %ld executing execlp() ...\n", (long) getpid());
7      ret = execlp("/bin/echo", "echo", "Hello\n", NULL);
8      if (ret < 0) perror("Fail to call execlp...");
9      printf("World\n");
10     return ret;
11 }
```

这些代码段会被执行吗？

A. execlp() 调用成功

B. execlp() 调用失败



2.3

进程创建 – 2.3.2 程序替换：exec函数族



```
1  #include <unistd.h>
2  #include <stdio.h>
3  pid_t Fork(void);
4  int main(int argc, char *argv[])
5  {
6      pid_t cid;
7      for (int i = 0; i < 2; i++){
8          cid = Fork();
9          if (cid == 0) execlp("/bin/echo","echo","Hello\n",NULL);
10     }
11     printf("*\n");
12 }
```

请思考这个代码的运行结果，并上机验证



```
1  #include <unistd.h>
2  #include <stdio.h>
3  pid_t Fork(void);
4  int main(int argc, char *argv[])
5  {
6      for (int i = 0; i < 2; i++){
7          Fork();
8          printf("*");
9      }
10     return 0;
11 }
```

请问这个程序运行过程中

1) 生成的进程数（含初始进程）？

2) 这段代码一共输出多少个字符 '*'

4个进程

8个 '*' ?

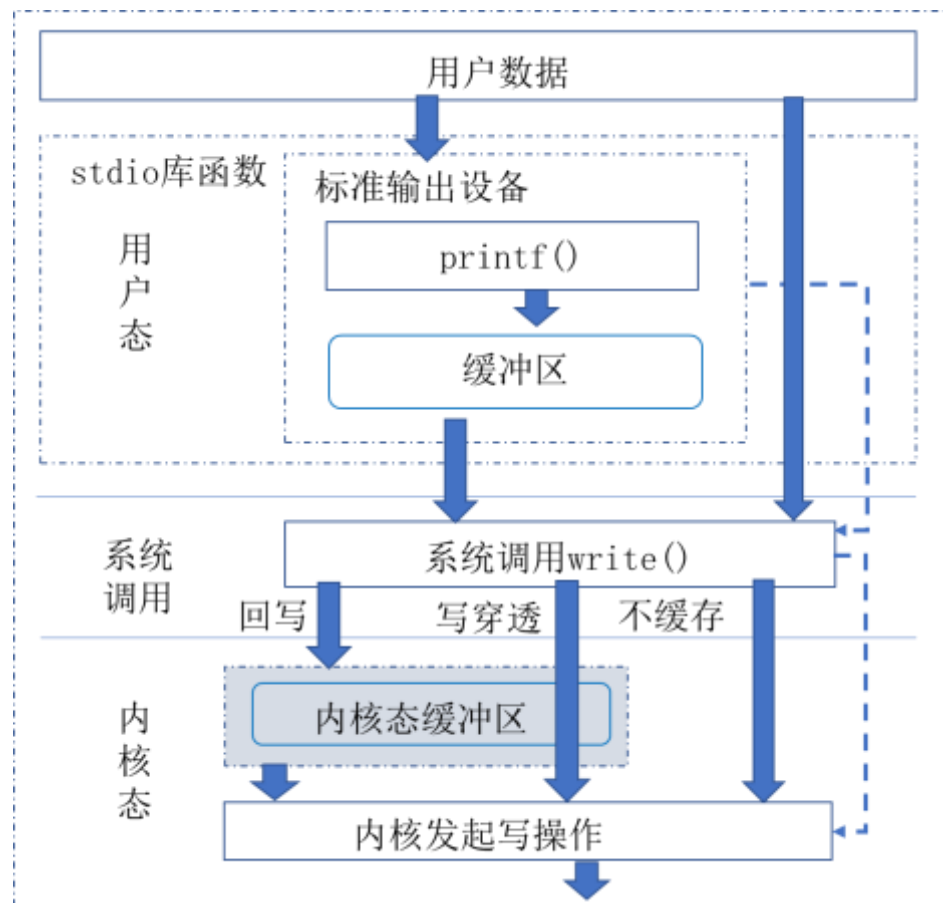


2.3

进程创建 – 2.3.3 资源继承机制



```
1  #include <unistd.h>
2  #include <stdio.h>
3  pid_t Fork(void);
4  int main(int argc, char *argv[])
5  {
6      for (int i = 0; i < 2; i++){
7          Fork();
8          printf("*");
9      }
10     return 0;
11 }
```



1. 输出缓冲机制

2. 缓冲区作为资源被子进程继承

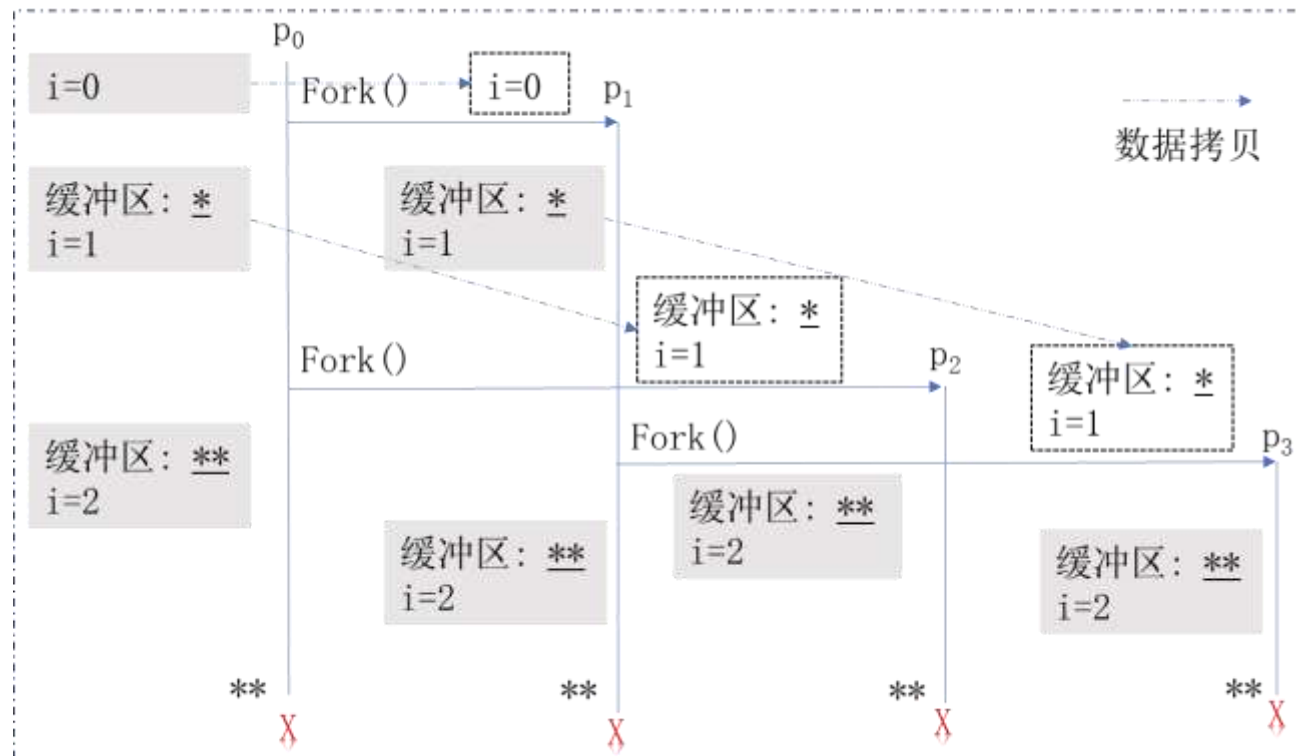


2.3

进程创建 – 2.3.3 资源继承机制



```
1  #include <unistd.h>
2  #include <stdio.h>
3  pid_t Fork(void);
4  int main(int argc, char *argv[])
5  {
6      for (int i = 0; i < 2; i++){
7          Fork();
8          printf("*");
9      }
10     return 0;
11 }
```



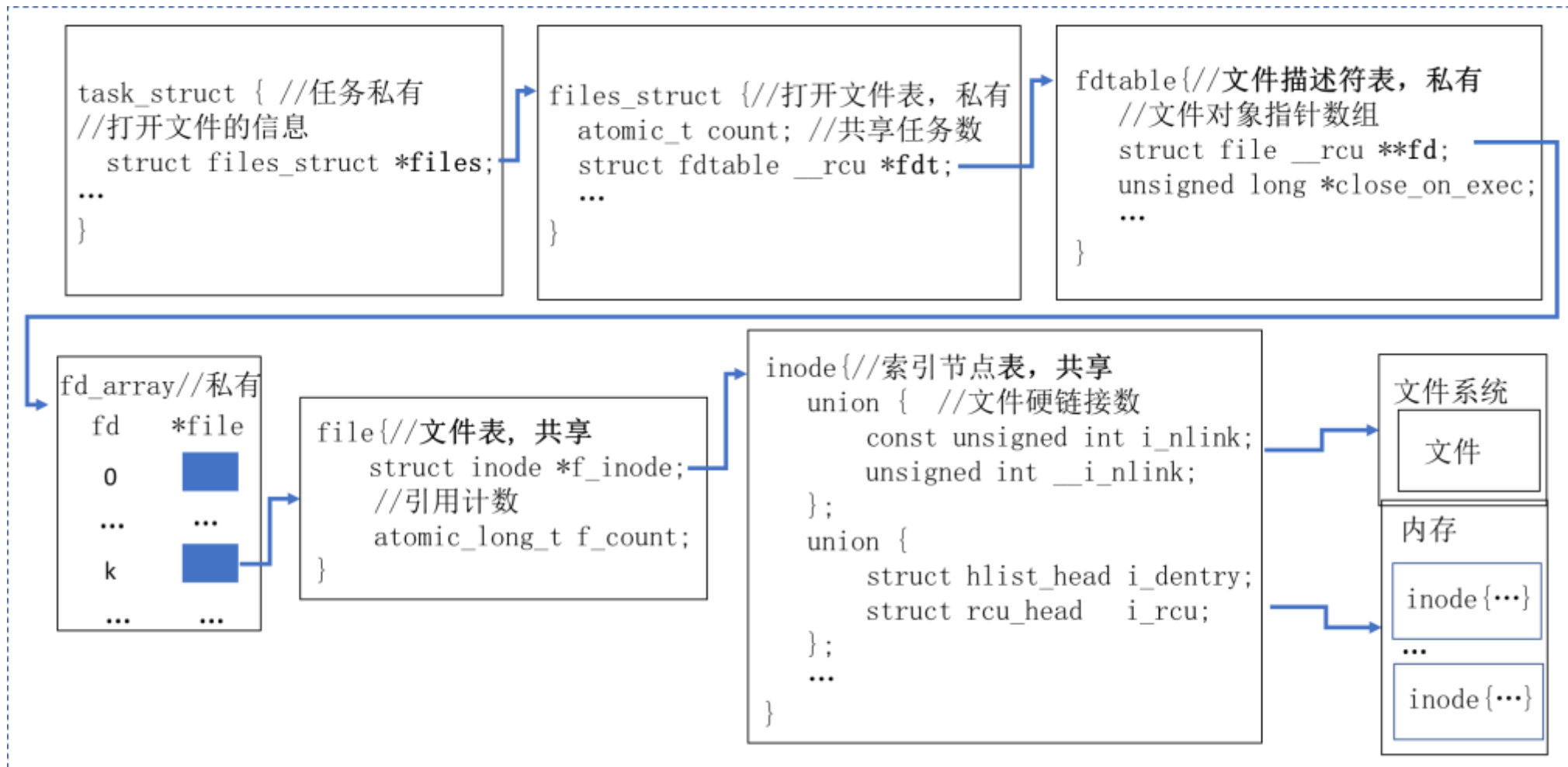


2.3

进程创建 - 2.3.3 资源继承机制



文件管理相关的内核数据结构节选





2.3

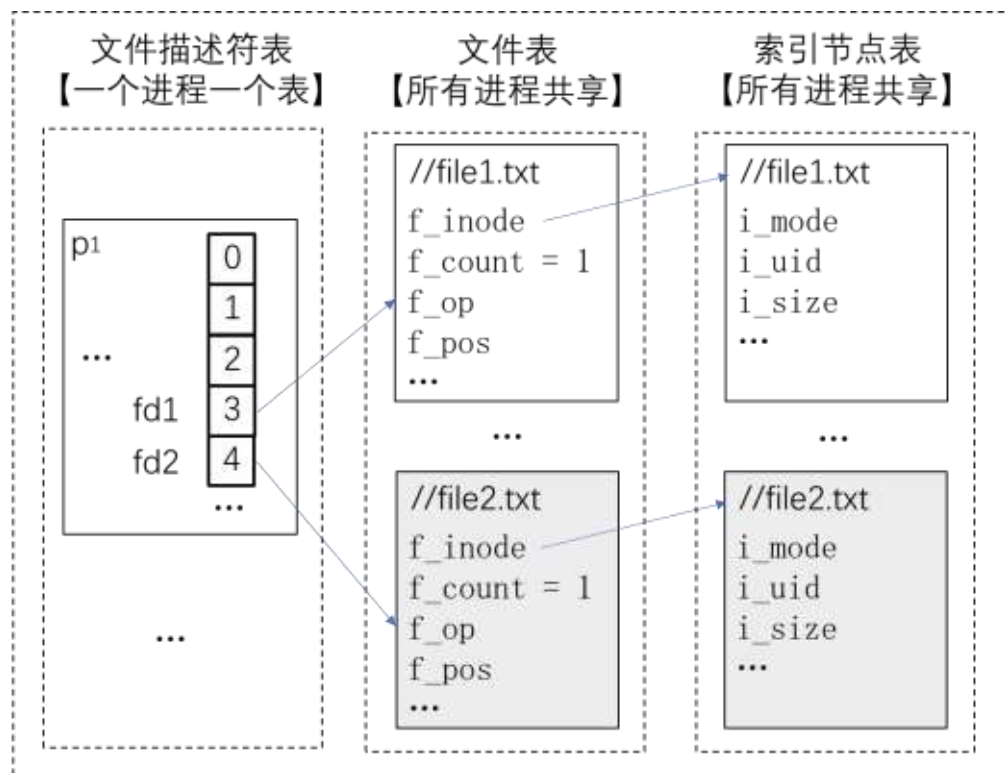
进程创建 – 2.3.3 资源继承机制



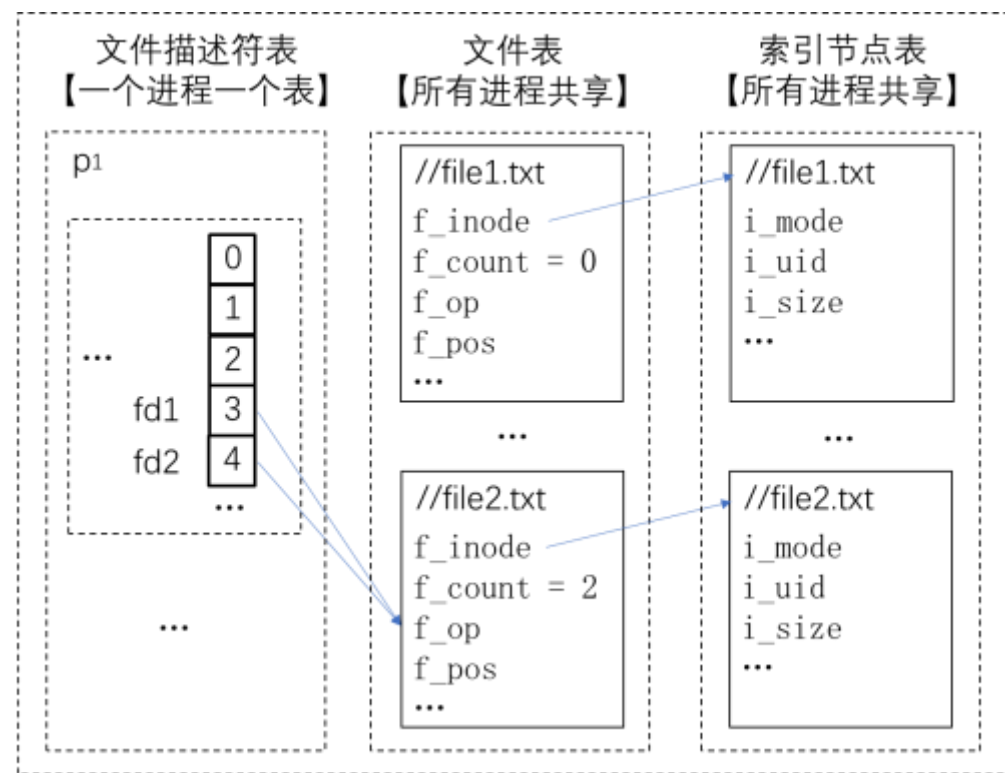
```
#include <unistd.h>
```

```
int dup(int old);      int dup2(int oldfd, int newfd)
```

打开文件file1.txt, file2.txt



调用
dup2(4,3)



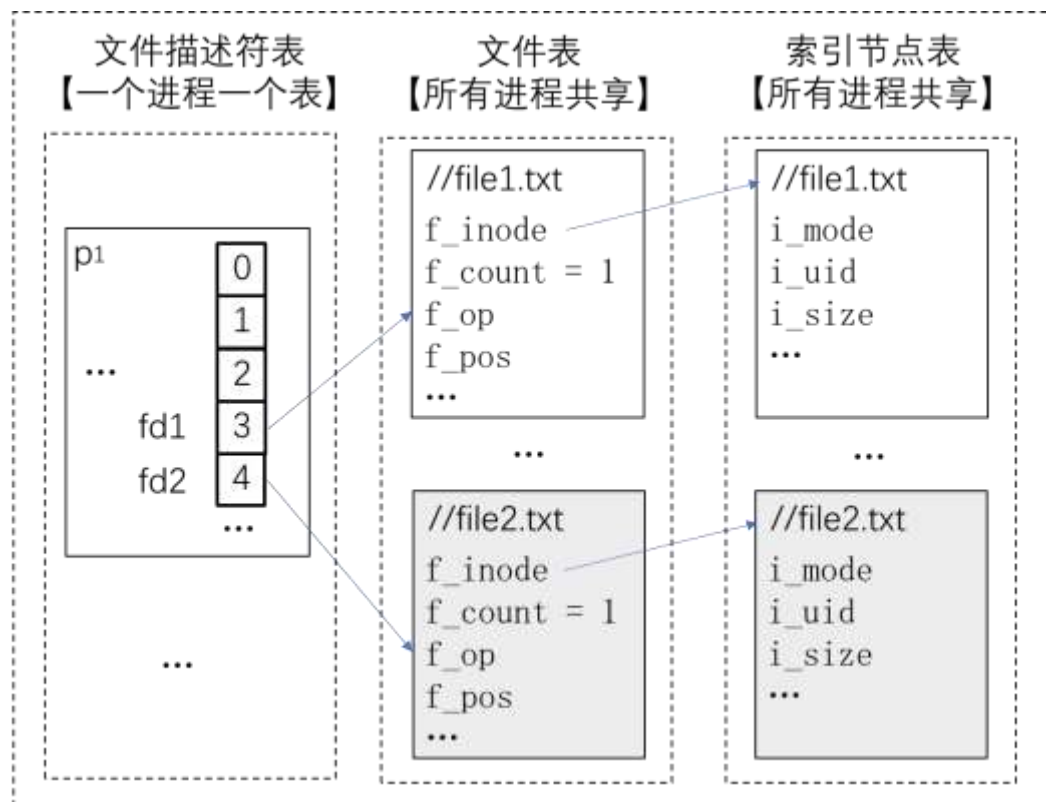


2.3

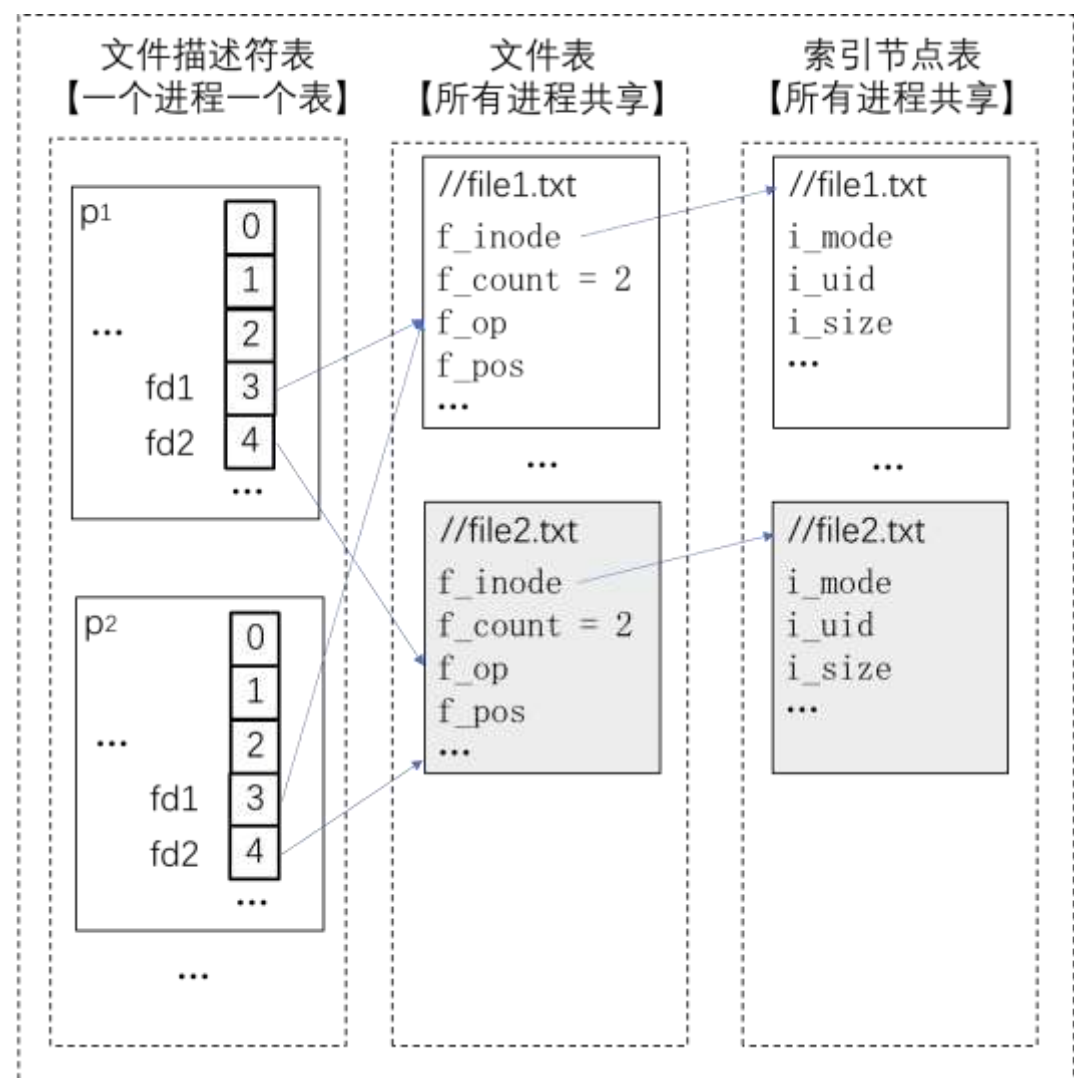
进程创建 - 2.3.3 资源继承机制



打开文件file1.txt, file2.txt



调用
`fork()`



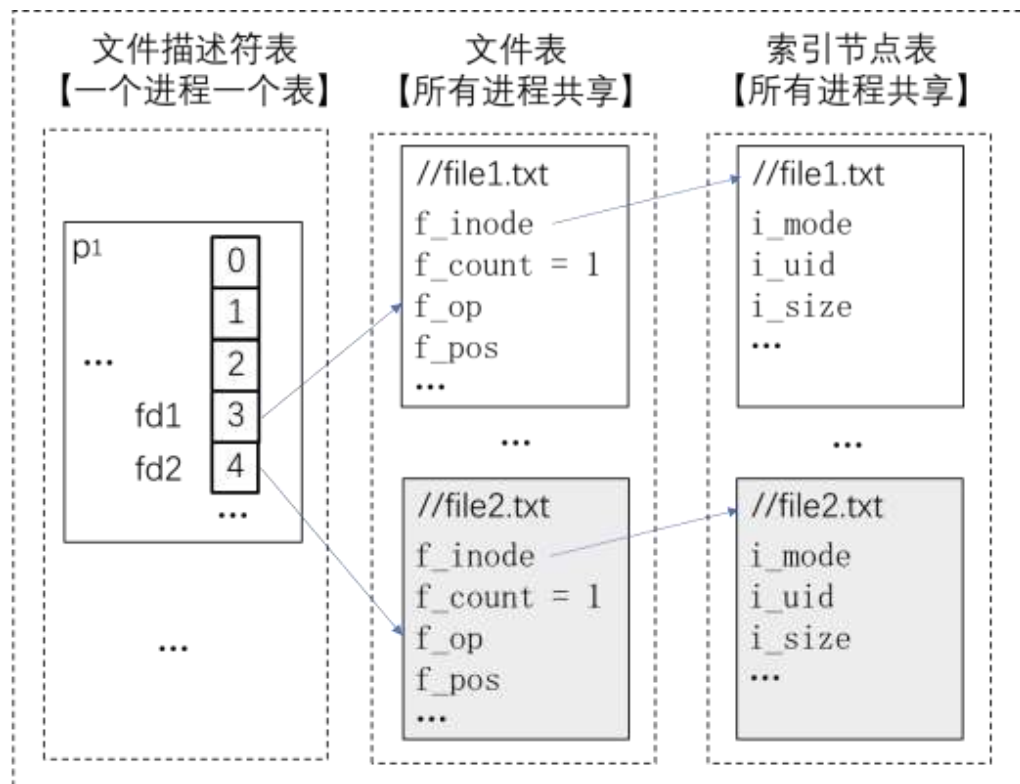


2.3

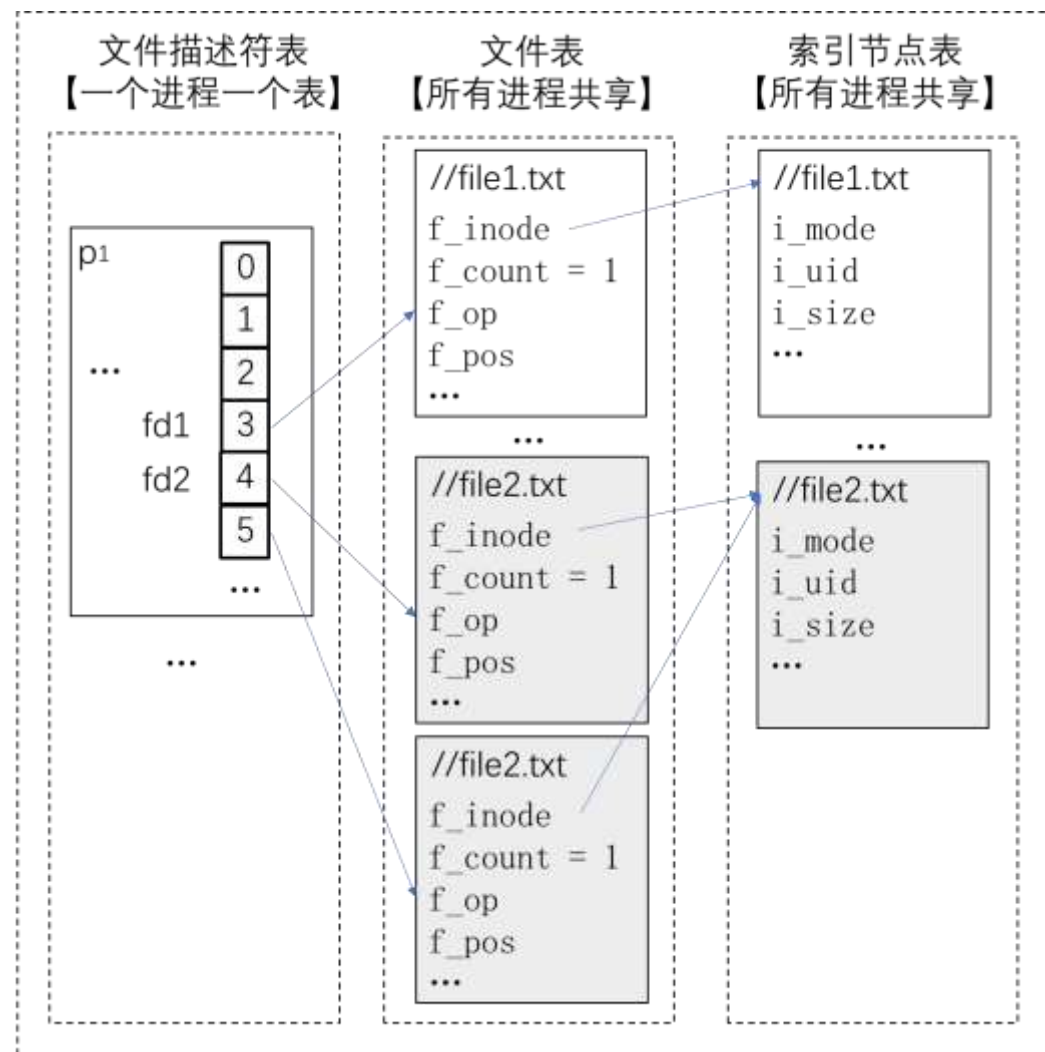
进程创建 - 2.3.3 资源继承机制



打开文件file1.txt, file2.txt



再次
打开文件
file2.txt





2.3

进程创建 – 2.3.3 资源继承机制



```
1  #include <stdio.h>
2  #include <unistd.h>
3  int main(int argc, char *argv[])
4  {
5      int ret = 0;
6      printf("My Id is %ld",getpid());
7      if ((ret=fork())>0){
8          printf("...\n");
9      } else ret = execl("/bin/ls","ls","-l",".",NULL);
10 }
```

课本P81 Q2.3-8

请思考这个代码的运行结果，并上机验证



2.3

进程创建 – 2.3.3 资源继承机制



```
1  #include <stdio.h>
2  #include <unistd.h>
3  void print_str(char str[])
4  {
5      char buf[50];
6      setbuf(stdout,buf);
7      printf("%s",str);
8  }
9  int main()
10 {
11     print_str("1 ");
12     print_str("2 ");
13     print_str( "3 ");
14 }
```

课本P81 Q2.3-9

请问，这个代码存在什么问题？请解释



正常终止方式

❖ 显式的系统调用

```
#include <unistd.h>
void _exit(int status);
```

❖ 显式的函数调用

```
#include <stdlib.h>
void exit(int status);
```

❖ return

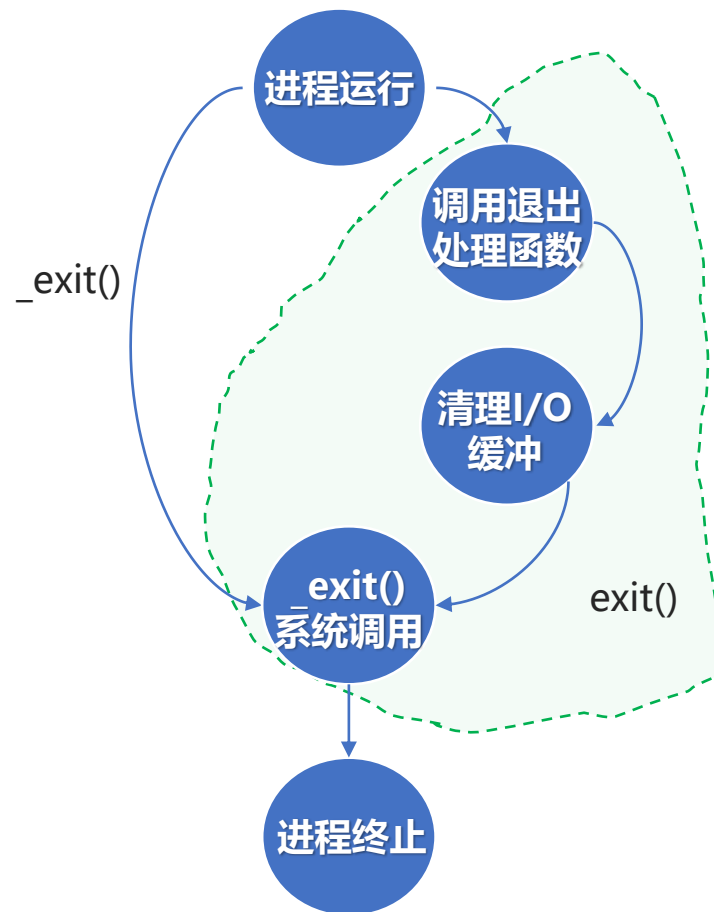
❖ 从程序的结尾离开

```
/* /usr/include/stdlib.h */
#define EXIT_FAILURE 1 /* 异常退出 */
#define EXIT_SUCCESS 0 /* 正常退出 */
```

退出处理函数

```
#include <stdlib.h>
//标准C库函数
int atexit(void (*function)(void));

#include <stdlib.h>
int on_exit(void (*function)(int , void *), void *arg);
```





2.4

进程终止 – 2.4.1 正常终止



```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <sys/types.h>
4  #include <unistd.h>
5  pid_t Fork(void);
6  void cleanup_wb(void) { printf("Proc %ld cleanup_wb\n", (long) getpid()); }
7  void cleanup_unlock(void) { printf("Proc %ld cleanup_unlock\n", (long) getpid()); }
8  int main(int argc, char *argv[])
9  {
10     atexit(cleanup_unlock);
11     atexit(cleanup_wb);
12     Fork();
13     exit(EXIT_SUCCESS);
14 }
```

\$/a.out

Proc 78347 cleanup_wb

Proc 78347 cleanup_unlock

Proc 78348 cleanup_wb

Proc 78348 cleanup unlock

退出处理函数管理采用后进先出 (LIFO) 原则：后注册者，先被调用



异常终止方式

❖ 主动终止

```
#include <stdlib.h>
void abort(void);
```

❖ 被动终止

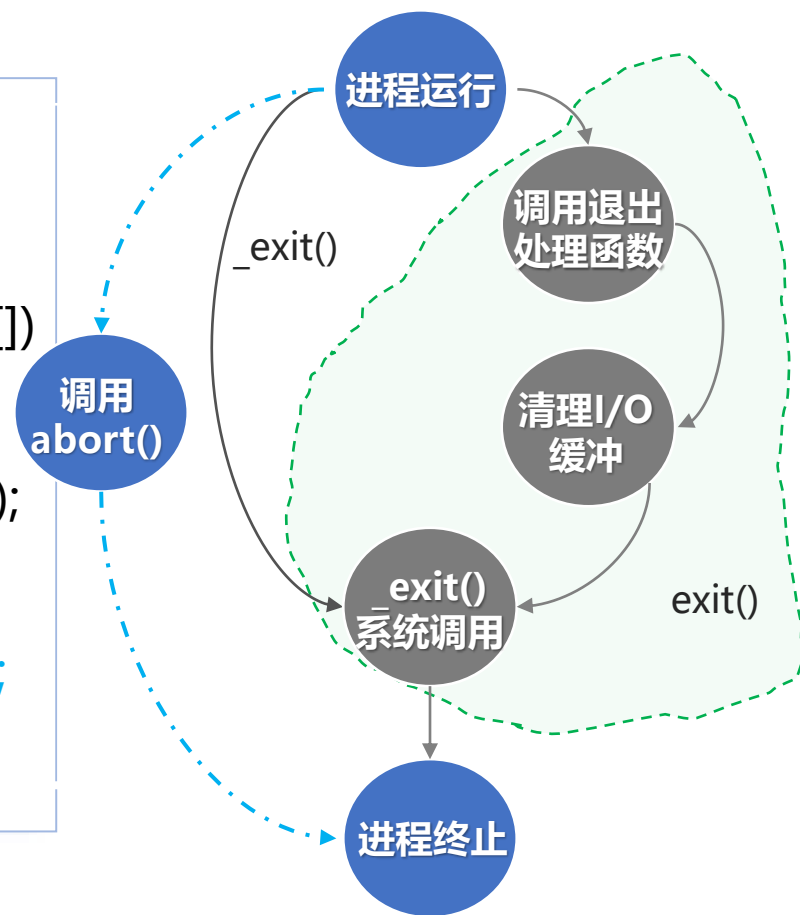
❖ Shell命令引发信号

❖ kill/pkill/...

❖ 内核自动触发

❖ 段错误/除零错误/...

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main(int argc, char *argv[])
4 {
5     printf("Calling abort().\n");
6     abort();
7     printf("Finished abort() ");
8 }
```





2.4

进程终止 – 2.4.2 异常终止



异常终止方式

❖ 主动终止

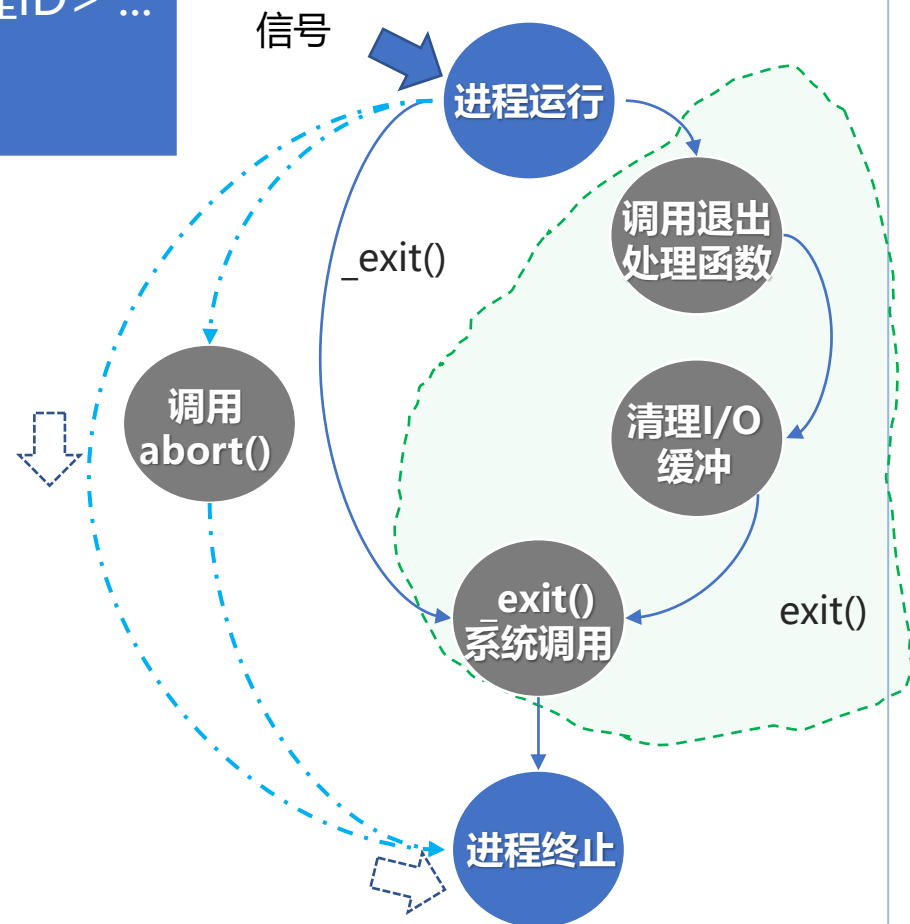
```
#include <stdlib.h>
void abort(void);
```

❖ 被动终止

- ❖ Shell命令引发信号
 - ❖ kill/pkill/killall...
- ❖ 内核自动触发
 - ❖ 段错误/除零错误/...

```
kill [-s <信号名或编号>] <进程ID> ...
kill -l [<信号编号>]
```

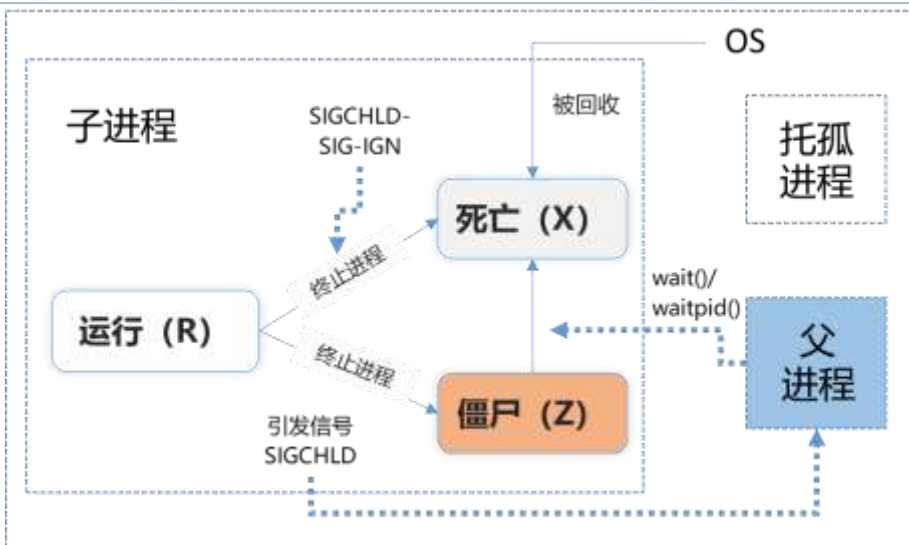
```
# 终止 PID 为 324的进程
$ kill -9 324
# 使用信号名称终止进程
$ kill -s SIGKILL 324
# 查看所有信号列表
$ kill -l
# 查看编号为 9 的信号名称
$ kill -l 9
```



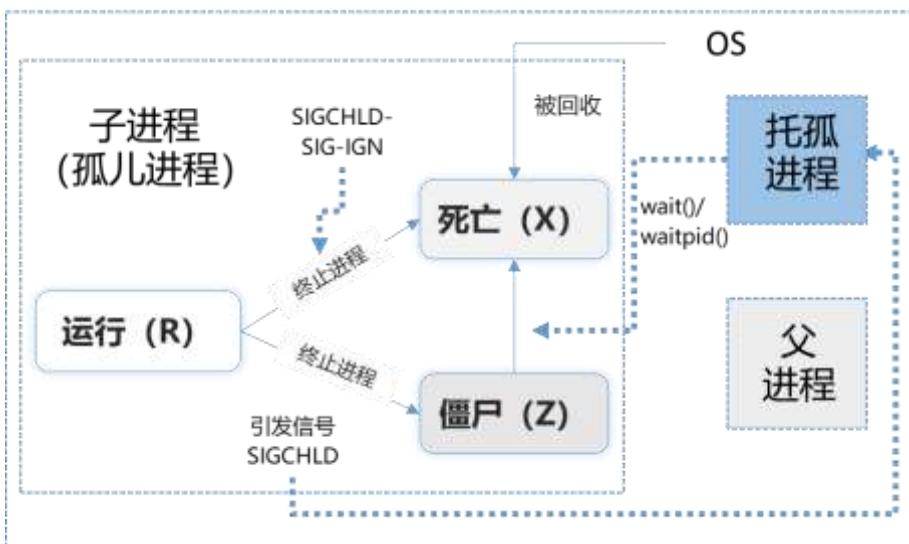


2.4

进程终止 – 2.4.3 孤儿进程与僵尸进程



```
1 int main(int argc, char *argv[])
2 {
3     if (!Fork()) { abort(); }
4     while(1);
5 }
```



```
1 int main(int argc, char *argv[])
2 {
3     if (Fork()) { exit(EXIT_SUCCESS); }
4     sleep(10);
5 }
```

僵尸进程内核结构 (task_struct)

- nKB
- 已释放
 - ✓ 用户地址空间
 - ✓ 打开文件表
 - ✓ 文件描述符
 - ✓ 上下文

- ...
- 依然保留
 - ✓ pid
 - ✓ state
 - ✓ tgid
 - ✓ utime
 - ✓ stime
 - ✓ parent/children/...
- ...

孤儿进程：无害，系统自动回收 | 僵尸进程：少量无害，大量耗尽pid，系统崩溃